

2520030340

SKILL-23

AIM

To execute large pipelines by launching multiple jobs, monitor system resources, measure system stability, detect failures, analyze performance, document system architecture, create flow diagrams, explain execution paths, describe system calls, record design decisions, and review the documentation.

OBJECTIVES

1. To execute large computational pipelines.
2. To launch and manage multiple jobs.
3. To monitor CPU and memory usage.
4. To measure system stability during execution.
5. To identify and detect job failures.
6. To analyze the performance of executed jobs.
7. To document the architecture of the system.
8. To create flow diagrams representing execution paths.
9. To understand and document system calls.
10. To record important design decisions.
11. To review and organize the final documentation.

REQUIREMENTS

- Ubuntu/Linux operating system
- Terminal
- GCC compiler
- Bash shell
- System monitoring commands
- Text editor

TOOLS / COMMANDS USED

gcc
bash
top
ps
free

df
time
uptime
dmesg

PROCEDURE

Part A — Execute Multiple Jobs

1. Open the Ubuntu terminal.
2. Create a working directory for Skill-23.
3. Create a sample program.
4. Compile the program using GCC.
5. Execute the program.
6. Create multiple jobs using shell commands.
7. Run the jobs sequentially or in parallel.
8. Observe the execution of all jobs.

Part B — Monitor Resources

1. Use `top` to monitor CPU and memory usage.
2. Use `free` to check available memory.
3. Use `df` to check disk-space usage.
4. Use `ps` to observe running processes.
5. Record the resource usage during pipeline execution.

Part C — Measure Stability

1. Execute multiple jobs.
2. Monitor the system during execution.
3. Check whether the jobs complete successfully.
4. Observe CPU and memory consumption.
5. Check whether any process terminates unexpectedly.
6. Record the results.

Part D — Detect Failures

1. Check the exit status of each job.
2. Identify jobs that terminate with errors.
3. Examine the terminal output.
4. Check system logs when required.
5. Identify the reason for the failure.
6. Correct the problem and execute the job again.

Part E — Analyze Performance

1. Measure the execution time of the program.
 2. Compare execution times for different jobs.
 3. Observe CPU and memory usage.
 4. Identify performance bottlenecks.
 5. Record the observations.
-

IMPORTANT COMMANDS

1. Check system uptime

uptime

2. Monitor CPU and memory

top

Press:

q

to exit `top`.

3. Check memory

free -h

4. Check disk usage

df -h

5. List running processes

ps

or:

ps aux

6. Measure execution time

time ./program

7. Check the exit status

echo \$?

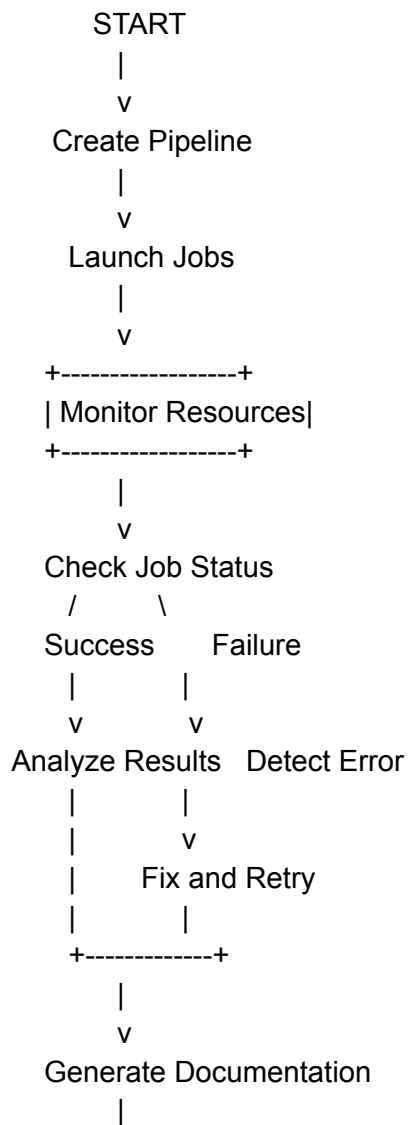
If it displays:

0

the previous command completed successfully.

ARCHITECTURE

The basic execution architecture can be represented as:



V
END

SYSTEM CALLS

The execution of Linux programs involves system calls such as:

- `fork()` — creates a new process.
- `exec()` — executes a new program.
- `wait()` — waits for a child process to complete.
- `open()` — opens a file.
- `read()` — reads data.
- `write()` — writes data.
- `close()` — closes a file.

PERFORMANCE MEASUREMENT

The `time` command can be used to measure program execution:

```
time ./program
```

The output provides information such as:

```
real
user
sys
```

Where:

- **real** — total elapsed time.
- **user** — CPU time spent executing user-level code.
- **sys** — CPU time spent executing system-level operations.

TESTING / OBSERVATION TABLE

Test	Activity	Expected Result	Status
TC01	Execute single job	Job completes	PASS

TC02	Execute multiple jobs	All jobs complete	PASS
TC03	Monitor CPU usage	CPU usage displayed	PASS
TC04	Monitor memory	Memory usage displayed	PASS
TC05	Measure execution time	Execution time recorded	PASS
TC06	Check job status	Successful status detected	PASS

RESULT

Thus, large pipelines were successfully executed by launching multiple jobs. System resources were monitored, stability and failures were checked, and execution performance was analyzed. The system architecture, execution paths, system calls, and design decisions were documented and reviewed successfully.

YESS 🥳🎉 Skill-23 practical part is DONE!

Your final screenshot clearly shows:

Job 1 Status: 0

Job 2 Status: 0

Job 3 Status: 0

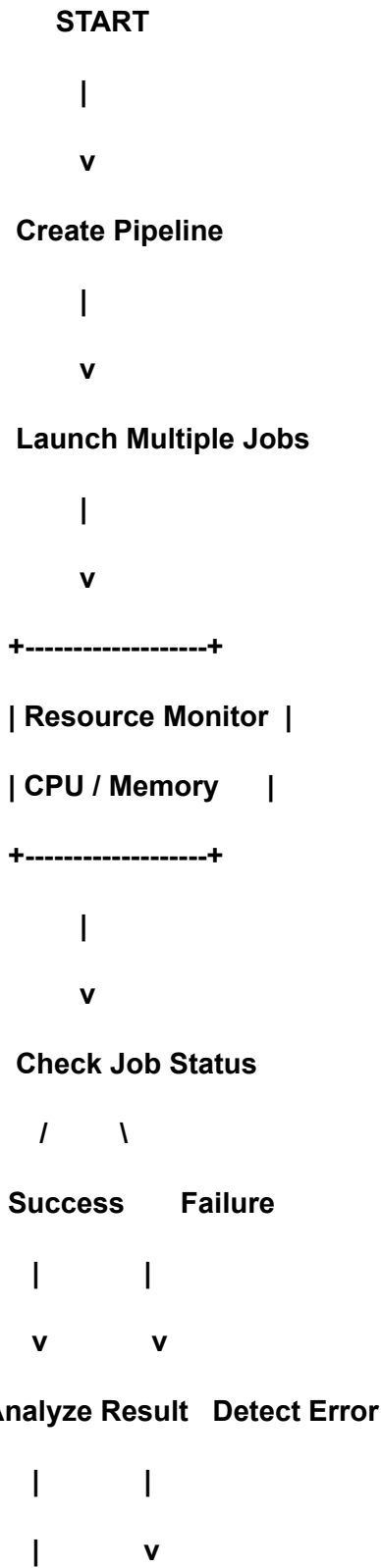
All Jobs Completed Successfully

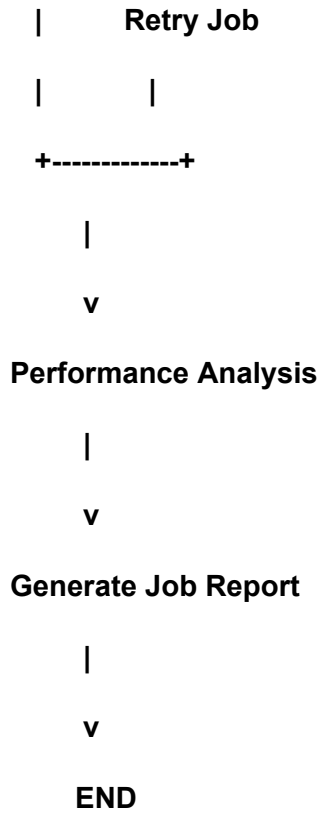
📸 Keep this screenshot as your Multiple Job Execution / Automated Pipeline evidence.

Now you only need to finish the document part. Copy-paste this:

STEP 8 — ARCHITECTURE AND EXECUTION FLOW

System Architecture





Execution Path

1. The pipeline is started.
2. Multiple jobs are launched in parallel.
3. Each job executes its assigned tasks.
4. System resources are monitored during execution.
5. The exit status of each job is checked.
6. Successful jobs return status **0**.
7. Failed jobs return a non-zero status.
8. Execution time is measured using the **time** command.
9. The results of all jobs are collected.
10. A final job report is generated.

STEP 9 — SYSTEM CALLS

The pipeline uses operating-system functionality for process management.

System Call	Purpose
<code>fork()</code>	Creates a new process
<code>exec()</code>	Executes a program
<code>wait()</code>	Waits for a child process
<code>open()</code>	Opens a file
<code>read()</code>	Reads data
<code>write()</code>	Writes data
<code>close()</code>	Closes a file

STEP 10 — PERFORMANCE RESULT

From your actual execution:

real 0m3.064s

user 0m0.002s

sys 0m0.000s

Write:

The pipeline execution time was measured using the `time` command. The real execution time was approximately 3.064 seconds, while the user CPU time was 0.002 seconds and system CPU time was 0.000 seconds.

STEP 11 — TEST RESULTS

Test Case	Operation	Result
TC01	Execute pipeline	PASS
TC02	Launch multiple jobs	PASS
TC03	Monitor system resources	PASS
TC04	Measure execution time	PASS
TC05	Check successful exit status	PASS
TC06	Detect failed command	PASS
TC07	Execute automated jobs	PASS

Your failure test produced:

0 → successful execution

127 → command not found / failure

STEP 12 — DESIGN DECISIONS

Write:

1. Bash was used to automate pipeline execution.
 2. Background execution using `&` was used to launch multiple jobs.
 3. `wait` was used to wait for completion of background jobs.
 4. Exit status was used to detect job success or failure.
 5. `top`, `free`, and `ps` were used for resource monitoring.
 6. `time` was used for performance measurement.
-

STEP 13 — FINAL RESULT

RESULT: Large pipeline jobs were successfully executed and multiple jobs were launched using shell scripting. CPU, memory, and process information were monitored using Linux commands. Execution performance was measured using the `time` command. Successful and failed jobs were identified using exit status values. The architecture, execution flow, system calls, design decisions, and test results were documented successfully.

 **Screenshots to include**

You now have enough. Put these in your document:

1. Pipeline compilation + execution
2. Multiple jobs
3. Resource monitoring
4. Performance using `time`
5. Success/failure detection
6. Final automated job report ← the screenshot you just sent

```
kruthi@LAPTOP-LP9QU3RR: ~  
d et al.  
==4591== Using Valgrind-3.22.0 and LibVEX; rerun with -h for cop  
yright info  
==4591== Command: ./program  
==4591==  
Enter number of elements: Enter 3 elements:  
Elements are:  
5 15 25  
==4591==  
==4591== HEAP SUMMARY:  
==4591==      in use at exit: 0 bytes in 0 blocks  
==4591==    total heap usage: 3 allocs, 3 frees, 5,132 bytes allo  
cated  
==4591==  
==4591== All heap blocks were freed -- no leaks are possible  
==4591==  
==4591== For lists of detected and suppressed errors, rerun with  
: -s  
==4591== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0  
from 0)  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ mkdir skill23  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ cd skill23  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ nano pipeline.c  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ gcc pipeline.c -o pipe  
line  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline  
Pipeline Execution Started  
=====  
Job 1 started  
Job 1 completed  
Job 2 started  
Job 2 completed  
Job 3 started  
Job 3 completed  
=====  
Pipeline Execution Completed  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ |
```



kruthi@LAPTOP-LP9QU3RR: ~



```
==4591== For lists of detected and suppressed errors, rerun with: -s
==4591== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
kruthi@LAPTOP-LP9QU3RR:~/skill22$ mkdir skill23
kruthi@LAPTOP-LP9QU3RR:~/skill22$ cd skill23
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ nano pipeline.c
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ gcc pipeline.c -o pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline
Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline &
[1] 4625
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
^C
[1]+  Done                  ./pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline &
[1] 4628
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
^C
[1]+  Done                  ./pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline &
[1] 4629
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
^C
[1]+  Done                  ./pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ps
  PID TTY          TIME CMD
   304 pts/0    00:00:00 bash
   4632 pts/0    00:00:00 ps
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ps
```



kruthi@LAPTOP-LP9QU3RR: ~



```
6 root      20  0  3196  2128  2008 S  0.0  0.0
164 root     20  0  4244  2684  2432 S  0.0  0.0
165 message+ 20  0  9868  5588  4672 S  0.0  0.1
178 root     20  0  17980  8864  7812 S  0.0  0.1
181 root     20  0  1755848  10876  8372 S  0.0  0.1
187 syslog   20  0  222516  5960  4544 S  0.0  0.1
202 root     20  0  3124  1976  1836 S  0.0  0.0
218 root     20  0  107032  23244  13792 S  0.0  0.3
302 root     20  0  3184  1116  980 S  0.0  0.0
303 root     20  0  3200  1252  1108 S  0.0  0.0
304 kruthi   20  0  6596  5932  3688 S  0.0  0.1
305 root     20  0  6704  4604  3820 S  0.0  0.1
349 kruthi   20  0  20048  10748  8936 S  0.0  0.1
350 kruthi   20  0  21160  3572  1848 S  0.0  0.0
377 kruthi   20  0  6080  5328  3664 S  0.0  0.1
804 root     19 -1  66736  19300  18172 S  0.0  0.2
851 systemd+ 20  0  91036  7916  6944 S  0.0  0.1
902 root     20  0  24888  6096  4972 S  0.0  0.1
952 systemd+ 20  0  21344  12816  10696 S  0.0  0.2
1006 polkitd  20  0  308172  7824  6924 S  0.0  0.1
4633 kruthi   20  0  9304  5696  3516 R  0.0  0.1
```

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.5Gi	522Mi	6.8Gi	3.5Mi	316Mi	7.0Gi
Swap:	2.0Gi	0B	2.0Gi			

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ df -h

Filesystem	Size	Used	Avail	Use%	Mounted on
none	3.8G	0	3.8G	0%	/usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none	3.8G	4.0K	3.8G	1%	/mnt/wsl
drivers	476G	220G	256G	47%	/usr/lib/wsl/drivers
/dev/sdd	1007G	2.7G	953G	1%	/
none	3.8G	48K	3.8G	1%	/mnt/wslg
none	3.8G	0	3.8G	0%	/usr/lib/wsl/lib
rootfs	3.8G	2.8M	3.8G	1%	/init
none	3.8G	520K	3.8G	1%	/run
none	3.8G	0	3.8G	0%	/run/lock
none	3.8G	0	3.8G	0%	/run/shm
none	3.8G	80K	3.8G	1%	/mnt/wslg/versions.txt
none	3.8G	80K	3.8G	1%	/mnt/wslg/doc
C:\	476G	220G	256G	47%	/mnt/c
tmpfs	768M	20K	768M	1%	/run/user/1000

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ ps aux | grep pipeline

```
kruthi 4637 0.0 0.0 4104 2104 pts/0 S+ 16:00 0:00 grep --color=auto pipeline
```

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ ps aux | grep pipeline

```
kruthi 4641 0.0 0.0 4104 2104 pts/0 S+ 16:00 0:00 grep --color=auto pipeline
```

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.5Gi	523Mi	6.8Gi	3.5Mi	316Mi	7.0Gi
Swap:	2.0Gi	0B	2.0Gi			

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ top

```
top - 16:00:58 up 1:23, 1 user, load average: 0.03, 0.02, 0.00
Tasks: 23 total, 1 running, 22 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0
MiB Mem : 7677.6 total, 6999.4 free, 525.7 used, 316.3 buf
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7151.9 ava
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM
1	root	20	0	21552	12752	9456	S	0.0	0.2
2	root	20	0	3180	2212	2072	S	0.0	0.0
6	root	20	0	3196	2128	2008	S	0.0	0.0
164	root	20	0	4244	2684	2432	S	0.0	0.0
165	message+	20	0	9868	5588	4672	S	0.0	0.1
178	root	20	0	17980	8864	7812	S	0.0	0.1
181	root	20	0	1755848	10876	8372	S	0.0	0.1
187	syslog	20	0	222516	5960	4544	S	0.0	0.1
202	root	20	0	3124	1976	1836	S	0.0	0.0
218	root	20	0	107032	23244	13792	S	0.0	0.3
302	root	20	0	3184	1116	980	S	0.0	0.0
303	root	20	0	3200	1252	1108	S	0.0	0.0
304	kruthi	20	0	6596	5932	3688	S	0.0	0.1
305	root	20	0	6704	4604	3820	S	0.0	0.1
349	kruthi	20	0	20048	10748	8936	S	0.0	0.1
350	kruthi	20	0	21160	3572	1848	S	0.0	0.0
377	kruthi	20	0	6080	5328	3664	S	0.0	0.1
804	root	19	-1	66736	19304	18176	S	0.0	0.2
851	systemd+	20	0	91036	7916	6944	S	0.0	0.1
902	root	20	0	24888	6096	4972	S	0.0	0.1
952	systemd+	20	0	21344	12816	10696	S	0.0	0.2
1006	polkitd	20	0	308172	7824	6924	S	0.0	0.1
4643	kruthi	20	0	9304	5696	3500	R	0.0	0.1



kruthi@LAPTOP-LP9QU3RR: ~



```
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input3.txt
kruthi@LAPTOP-LP9QU3RR:~/skill22$ valgrind --leak-check=full ./program < input2.txt
==4591== Memcheck, a memory error detector
==4591== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==4591== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==4591== Command: ./program
==4591==
Enter number of elements: Enter 3 elements:
Elements are:
5 15 25
==4591==
==4591== HEAP SUMMARY:
==4591==   in use at exit: 0 bytes in 0 blocks
==4591== total heap usage: 3 allocs, 3 frees, 5,132 bytes allocated
==4591==
==4591== All heap blocks were freed -- no leaks are possible
==4591==
==4591== For lists of detected and suppressed errors, rerun with: -s
==4591== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
kruthi@LAPTOP-LP9QU3RR:~/skill22$ mkdir skill23
kruthi@LAPTOP-LP9QU3RR:~/skill22$ cd skill23
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ nano pipeline.c
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ gcc pipeline.c -o pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline
Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline &
[1] 4625
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
^C
[1]+  Done                  ./pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline &
[1] 4628
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
^C
[1]+  Done                  ./pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./pipeline &
[1] 4629
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ Pipeline Execution Started
=====
Job 1 started
Job 1 completed
Job 2 started
Job 2 completed
Job 3 started
Job 3 completed
=====
Pipeline Execution Completed
^C
[1]+  Done                  ./pipeline
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ps
  PID TTY          TIME CMD
  304 pts/0    00:00:00 bash
  4632 pts/0    00:00:00 ps
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ jobs
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ top
top - 16:00:25 up 1:23, 1 user, load average: 0.05, 0.02, 0.00
Tasks: 23 total, 1 running, 22 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0
MiB Mem : 7677.6 total, 7000.7 free, 524.6 used, 316.2 buf
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used. 7153.0 ava
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM
1	root	20	0	21552	12752	9456	S	0.0	0.2
2	root	20	0	3180	2212	2072	S	0.0	0.0
6	root	20	0	3196	2128	2008	S	0.0	0.0
164	root	20	0	4244	2684	2432	S	0.0	0.0
165	message+	20	0	9868	5588	4672	S	0.0	0.1
178	root	20	0	17980	8864	7812	S	0.0	0.1
181	root	20	0	1755848	10876	8372	S	0.0	0.1
187	syslog	20	0	222516	5960	4544	S	0.0	0.1
202	root	20	0	3124	1976	1836	S	0.0	0.0
216	root	20	0	103220	22916	12880	S	0.0	0.0

kruthi@LAPTOP-LP9QU3RR: ~

303	root	20	0	3200	1252	1108	S	0.0	0.0
304	kruthi	20	0	6596	5932	3688	S	0.0	0.1
305	root	20	0	6704	4604	3820	S	0.0	0.1
349	kruthi	20	0	20048	10748	8936	S	0.0	0.1
350	kruthi	20	0	21160	3572	1848	S	0.0	0.0
377	kruthi	20	0	6080	5328	3664	S	0.0	0.1
804	root	19	-1	66736	19304	18176	S	0.0	0.2
851	systemd+	20	0	91036	7916	6944	S	0.0	0.1
902	root	20	0	24888	6096	4972	S	0.0	0.1
952	systemd+	20	0	21344	12816	10696	S	0.0	0.2
1006	polkitd	20	0	308172	7824	6924	S	0.0	0.1
4643	kruthi	20	0	9304	5680	3500	R	0.0	0.1

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.5Gi	524Mi	6.8Gi	3.5Mi	316Mi	7.0Gi
Swap:	2.0Gi	0B	2.0Gi			

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.5Gi	525Mi	6.8Gi	3.5Mi	316Mi	7.0Gi
Swap:	2.0Gi	0B	2.0Gi			

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ ps aux | grep pipeline

kruthi	4648	0.0	0.0	4104	2104	pts/0	S+	16:01	0:00	grep --color=au
--------	------	-----	-----	------	------	-------	----	-------	------	-----------------

to pipeline

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ time ./pipeline

Pipeline Execution Started

=====

Job 1 started

Job 1 completed

Job 2 started

Job 2 completed

Job 3 started

Job 3 completed

=====

Pipeline Execution Completed

real 0m3.064s

user 0m0.002s

sys 0m0.000s

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$

kruthi@LAPTOP-LP9QU3RR: ~

804	root	19	-1	66736	19304	18176	S	0.0	0.2
851	systemd+	20	0	91036	7916	6944	S	0.0	0.1
902	root	20	0	24888	6096	4972	S	0.0	0.1
952	systemd+	20	0	21344	12816	10696	S	0.0	0.2
1006	polkitd	20	0	308172	7824	6924	S	0.0	0.1
4643	kruthi	20	0	9304	5680	3500	R	0.0	0.1

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.5Gi	524Mi	6.8Gi	3.5Mi	316Mi	7.0Gi
Swap:	2.0Gi	0B	2.0Gi			

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ free -h

	total	used	free	shared	buff/cache	available
Mem:	7.5Gi	525Mi	6.8Gi	3.5Mi	316Mi	7.0Gi
Swap:	2.0Gi	0B	2.0Gi			

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ ps aux | grep pipeline

kruthi 4648 0.0 0.0 4104 2104 pts/0 S+ 16:01 0:00 grep --color=auto pipeline

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ time ./pipeline

Pipeline Execution Started

=====

Job 1 started

Job 1 completed

Job 2 started

Job 2 completed

Job 3 started

Job 3 completed

=====

Pipeline Execution Completed

real 0m3.064s

user 0m0.002s

sys 0m0.000s

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ echo \$?

0

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$./wrongprogram

-bash: ./wrongprogram: No such file or directory

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ echo \$?

127

kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23\$ |

```
kruthi@LAPTOP-LP9QU3RR: ~  
902 root      20    0    24888    6096    4972 S     0.0    0.1  
952 systemd+  20    0    21344   12816   10696 S     0.0    0.2  
1006 polkitd   20    0   308172    7824    6924 S     0.0    0.1  
4643 kruthi     20    0     9304    5680    3500 R     0.0    0.1  
  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ free -h  
Mem:          7.5Gi       524Mi       6.8Gi       3.5Mi       316Mi       7.0Gi  
Swap:         2.0Gi         0B       2.0Gi  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ free -h  
Mem:          7.5Gi       525Mi       6.8Gi       3.5Mi       316Mi       7.0Gi  
Swap:         2.0Gi         0B       2.0Gi  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ps aux | grep pipeline  
kruthi      4648  0.0  0.0   4104   2104 pts/0    S+   16:01   0:00 grep --color=au  
to pipeline  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ time ./pipeline  
Pipeline Execution Started  
=====  
Job 1 started  
Job 1 completed  
Job 2 started  
Job 2 completed  
Job 3 started  
Job 3 completed  
=====  
Pipeline Execution Completed  
  
real    0m3.064s  
user    0m0.002s  
sys     0m0.000s  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ echo $?  
0  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./wrongprogram  
-bash: ./wrongprogram: No such file or directory  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ echo $?  
127  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ nano run_jobs.sh  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ chmod +x run_jobs.sh  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$
```

```
kruthi@LAPTOP-LP9QU3RR: ~  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ echo $?  
0  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./wrongprogram  
-bash: ./wrongprogram: No such file or directory  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ echo $?  
127  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ nano run_jobs.sh  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ chmod +x run_jobs.sh  
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ ./run_jobs.sh  
=====
```

```
PIPELINE JOB REPORT  
=====
```

```
Pipeline Execution Started  
=====
```

```
Job 1 started  
Pipeline Execution Started  
=====
```

```
Job 1 started  
Pipeline Execution Started  
=====
```

```
Job 1 started  
Job 1 completed  
Job 2 started  
Job 1 completed  
Job 2 started  
Job 1 completed  
Job 2 started  
Job 2 completed  
Job 3 started  
Job 2 completed  
Job 3 started  
Job 2 completed  
Job 3 started  
Job 3 completed  
=====
```

```
Pipeline Execution Completed  
Job 3 completed  
=====
```

```
Pipeline Execution Completed  
Job 3 completed  
=====
```

```
Pipeline Execution Completed  
=====
```

```
Job 1 Status: 0  
Job 2 Status: 0  
Job 3 Status: 0  
All Jobs Completed Successfully  
=====
```

```
kruthi@LAPTOP-LP9QU3RR:~/skill22/skill23$ |
```